SENTINEL Lab 3 Brief: `libipc`

ECE 476 Engineering System Software • Fall 2026 • University of Victoria

Lab session	Friday, November 20, 2026
Prelab due	Wednesday, November 18, 2026, by midnight, on Brightspace. Required, with no marks[1]
Deliverable due	Wednesday, December 2, 2026, by midnight, on Brightspace: 4% of the grade, out of 80 marks
Oral exam	In the Lab 4 session, Dec 4: 2% of the grade, out of 20 marks[2]
Lab 3 total	6% of the course grade: the deliverable and the oral exam

This brief is the one paper of Lab 3. It states what no other paper and no repository file states, and a footnote names the place of each other fact. Read it with `src/libipc/include/sentinel/ipc.h`[3], which states each function: its steps in order, its errors and its log lines.

1 The task

Lab 3 builds `libipc`, the operations centre. In Lab 2 the centre handled each report itself, and one thread cannot keep up with SAT, which sends 2000 reports each second. Lab 3 adds a pool of *worker threads*. The main thread of the centre puts each report into a shared queue, and the workers take the reports out and apply them to the *track picture*: the shared record of what is out there. Lab 3 does the work of R3, thousands of contacts each second.[4] It does NOT protect the shared data. You measure what that costs, and that is R4,[5] which Lab 4 solves.

Write these three files in `src/libipc/src/`, and two tests (Section 5):

- `queue.c`: `queue_init`, `queue_destroy`, `queue_push`, `queue_pop` and `queue_count`.
- `track.c`: `track_update`.
- `pool.c`: `pool_create_deadline`, the worker loop, `pool_stop`, `pool_destroy`, `pool_total_processed` and `pool_total_late`.

The course gives the other files as source. Read them, and do not change them: `src/libipc/Makefile`, `ipc.h`, and these files in `src/libipc/given/`:

- `queue_stats.c` reads the state and the statistics of the queue.
- `track_table.c` holds the hash table of the track picture: its start, its search, its totals, and `track_evict_one`, which your `track_update` calls.
- `pool_watch.c` holds the instruments of the pool, the report deadline, and the hooks that your worker loop calls before and after each report.
- `associate.c`, `validate.c` and `work.c` are for Lab 4 and for the cost of a report.
- `demo_lab3.c` is the demo.[6] Its main thread is the centre of Lab 2: it calls your Lab 2 functions, and it pushes each report into the queue.

Lab 3 links your own Lab 1 and Lab 2 libraries. A fault there is a fault in Lab 3.

2 Lab 3 in SENTINEL

Lab 3 makes two more of the user stories[7] of the course true. It also shows the fault of R4: you measure it, and you do not correct it.

[1] The Lab 3 prelab, `docs/lab3/prelab3.pdf` in your repository. It is due before the Lab 3 session.

[2] The oral exam of Lab 3 is in the Lab 4 session. The Submission Guide, Section 3, `docs/submit/submit.pdf`, describes its format.

[3] `src/libipc/include/sentinel/ipc.h`, in your repository. It states what each function of Lab 3 does, in order, with each of its errors and its log lines.

[4] R3 is a requirement of SENTINEL: thousands of contacts arrive each second. The SENTINEL Lab Manual, Section 1.2, `docs/manual/sentinel_manual.pdf`, lists the six requirements.

[5] R4 is a requirement of SENTINEL: many workers write the same shared record, and must not corrupt it. The SENTINEL Lab Manual, Section 1.2, `docs/manual/sentinel_manual.pdf`, lists the six requirements.

[6] `src/libipc/given/demo_lab3.c`, in your repository: the given demo of Lab 3. Its first comment states its options, its two verdicts and its exit status.

[7] The user stories are in Section 5 of the ECE 476 Lab Documentation for SENTINEL, which the course gives you: seven states of the centre, from *i. Normal Operations* to *vii. Operator Decision.*

Story	What Lab 3 makes true	Your code	What you see
iv. Concurrent Processing	Eight workers take reports from one queue, while the main thread continues to read the sensors	`pool.c`, `queue.c`	While `make lab3` runs, the page[8] shows `8 workers on 1 processor, using 0.23 of 1`
v. Track Picture Updated	Each report updates the track of its contact, or creates it	`track_update`	The track picture on the map of the page, and `track ID updated from SAT` in the log
R4	With many workers on many processors, the totals are wrong, or the stop never ends	all three files	`shared data DISAGREES` or `THE WORKERS DID NOT STOP` at the end of the demo

Figure 1 shows the threads of the centre, and who writes each part. All the threads are in ONE process, so they share its memory. The queue and the track picture are in that memory, and no lock protects them.

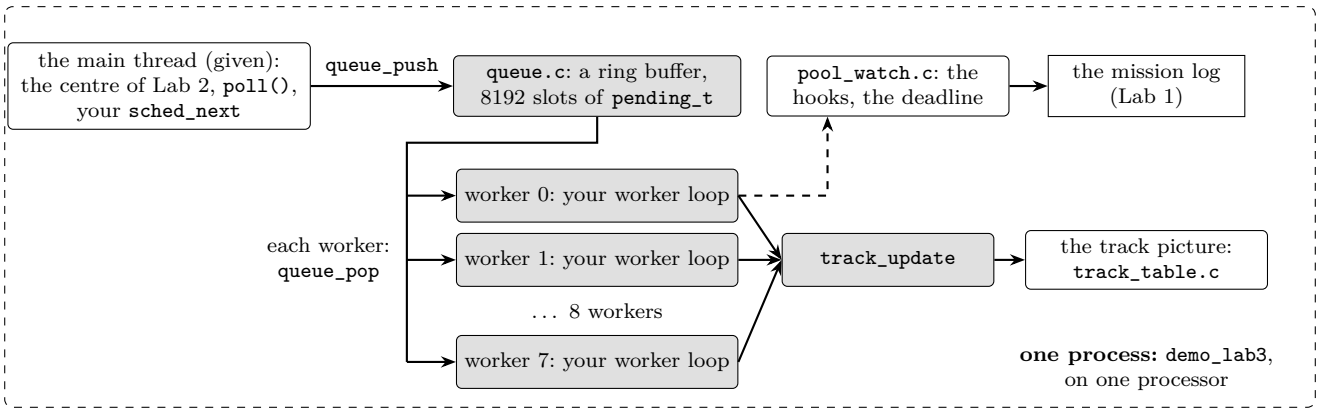


Figure 1. The threads of the centre in Lab 3. Grey: your code. White: given code. Square corners: a file. The queue and the track picture are shared, with no lock.

3 The ideas

3.1 A thread shares its process

A process can have many threads. Each thread runs the same program, and has its own stack and its own registers. All the threads of a process share its memory: its global variables and its heap. They also share its descriptors and its pid. So a thread sees each change that another thread makes to shared memory, with no pipe and no copy. `pthread_create()` starts a thread at a function, with one `void *` argument. `pthread_join()` waits until the thread returns. Compile and link with `-pthread`: the given `Makefile` does that.

3.2 Each worker has its own block

`pthread_create()` gives the new thread an ADDRESS. If each thread gets the address of the loop variable `i`, all threads read one variable that the loop changes, and each one reads the value that it has when that thread runs. So `pool_create_deadline` gives worker `i` the address of `workers[i]`: a block of its own, in the memory of the pool, that lives until `pool_destroy`. Only that worker writes its block, so the statistics in it are exact when the shared data is wrong.

3.3 The queue between the threads

The main thread is the *producer*: it pushes each report. The workers are the *consumers*: each one pops the next report. The queue is a ring buffer: `head` and `tail` move around a fixed array, and after the last slot they go back to

[8] The page is the operations centre display. Start it with `make watch`, and open `http://localhost:8080/`. `web/README.md`, in your repository, describes the page and each view.

slot 0. In a ring, `head == tail` when the queue is empty and also when it is full, so the queue keeps `count`. Each slot holds a `pending_t`[9], so a worker knows when the centre accepted the report.

A queue with a limit can be full. Then `queue_push` refuses the report with `ENOSPC`, and counts it in `rejected`. That is *back pressure*: the centre cannot take more than its workers apply, and the refused report is counted, not lost with no record. The demo prints each refused report in its LOSS LEDGER. At the default work of 200 μs for each report, the queue kept up with SAT[10] in each measured run on one processor: 21 reports of 8192 at most.

3.4 A stop that loses nothing

`pool_stop` sets `stop` to 1. Each worker reads `stop` in each turn of its loop. `stop` is `volatile`, so the compiler reads it from memory each time and does not keep it in a register. `volatile` is not a protection: ThreadSanitizer reports the write of `stop` against the reads, and that report is expected. After the stop, each worker takes each report that is still in the queue: the *drain*. Then it returns, and `pthread_join()` in `pool_stop` sees it. A worker whose queue is empty calls `usleep(200)`. A worker that tries again at once uses the processor, and on one processor it takes the processor from the main thread that fills the queue.

3.5 Three steps, not one

`t->total_reports++` is one line of C, and three steps for the processor: LOAD the value, ADD 1, STORE it. Two workers can both LOAD 5, and both STORE 6: one update is lost, and no error occurs. That is a *race*. Your `queue_pop` has the same three steps on `count`, and two workers can take the same report, or take 1 from `count` two times for one report. `count` is a `size_t`, so below 0 it becomes a very large number, the queue never becomes empty, and the drain never ends.

`demo_lab3 --workers 8 --ms 10000`	Runs	Agrees	DISAGREES	DID NOT STOP
`--cpus 1`, one processor	10	10	0	0
`--cpus 2`	10	0	2	8
`--cpus all`, 8 processors	10	0	3	7

Figure 2. The verdict of the demo in 30 runs of 10 s, measured on aarch64 with 8 processors. With `--workers 1`, each run agreed.

On one processor the race needs a context switch between the LOAD and the STORE of one worker, so it is rare: no run of Figure 2 showed it. With two processors or more, the workers run at the same time, and each run showed it. So a run that gives the right total proves nothing. ThreadSanitizer proves more: `-fsanitize=thread` makes the compiler add a check to each access to memory, and the program reports each access of two threads to the same place with no order between them, also when the total is right. It names the file and the line of each access, in your code.

3.6 Concurrency, not parallelism

The centre runs on one processor.[11] Its 8 workers are *concurrent*: many reports are in progress at the same time, and the workers take turns on the processor. They are not *parallel*: two of them never run at the same instant. So 8 workers get at most 1.00 of a processor in all, and 1000 reports of 200 μs take about 200 ms with 1 worker or with 8. The given instruments measure the processor time of each worker with `CLOCK_THREAD_CPUTIME_ID`, and the page draws the share of each.

[9] `pending_t` is the report that waits for the centre, from Lab 2: the contact, the time at which the centre accepted it, and its class. `src/libproc/include/sentinel/proc.h` states it.

[10] SAT sends in passes: 2000 reports each second for 10 s, then no report for 10 s. The SENTINEL Lab Manual, Section 1.3, `docs/manual/sentinel_manual.pdf`.

[11] The centre runs on one processor. `cpu_pin` and `cpu_unpin`, in `common/cpu.h`, set the processors that a process can use. The SENTINEL Lab Manual, Section 1.5, `docs/manual/sentinel_manual.pdf`, explains why.

4 The code rules

The code rules of the Lab 1 Brief[12] apply to Lab 3 too. `ipc.h`[13] gives the steps of each function, and of the worker loop under `pool_create_deadline`. The marks of the code rules (Section 7) check these steps. One rule is new: do not add a lock, an atomic variable or a semaphore. Lab 3 measures the code with no protection, and Lab 4 adds the protection.

5 Build, run and check

Lab 3 adds these targets to those of the SENTINEL Lab Manual.[14] Run each one at the top of your repository.

Command	What it does
`make`	Builds `src/libipc` with the given `Makefile`. Until your three files exist, it compiles the files that exist, names the others, and builds no library
`make lab3`	Runs `demo_lab3` with 8 workers for 3 s, for the page. `MS=20000` runs it for 20 s, `WORKERS=1` starts one worker, and `CPUS=all` gives the centre each processor
`make accept3`	Runs the 23 checks of `tools/accept/accept_lab3.c` on your library and the demo
`make lab3-tsan`	Builds `build/demo_lab3_tsan`: the demo, with ThreadSanitizer in the code of Lab 3
`make submit LAB=3`	Writes the zip[15] of Lab 3 (Section 6)

This is the end of the output of `make accept3` when each check passes:

```
queue.c          6 of 6   100%
track.c          6 of 6   100%
pool.c          10 of 10  100%
demo_lab3        1 of 1   100%

23 of 23 checks pass (100%).
```

A failed check prints a sentence about the contract. Read the check in `accept_lab3.c` to see what it does. The test defines `pthread_create()`: it holds each new worker until the check is ready, so each check that needs an exact total uses one worker, or fills the queue while no worker runs. Do not change the file: the marks come from the course copy of it.

Your tests. Write two test programs in `src/libipc/tests/`, as the Lab 1 Brief[16] describes. Each one uses ONE thread, and tests these cases. No more is necessary:

- `test_queue.c`: five reports come out in the order in which they went in; a queue of 4 slots after the nine operations of Q4 of the prelab; `ENOSPC` and `rejected` from a full queue; `EINVAL` for a NULL queue.
- `test_track.c`: a new id gives 1, and the same id again gives 0 with one entry in `track_count_entries`; three reports with the flag `P` for one id give `priority_tracks` 1.

`make test` builds and runs them. A failure of the kernel, such as a failed `malloc()`, needs no test, and no test with many threads is asked.

valgrind. `valgrind --leak-check=full build/demo_lab3 --workers 2 --ms 3000` checks the threads and the memory of the pool. `valgrind` runs one thread at a time, so the shared data agrees under it.

[12] The rules of the code of Lab 1, which apply to each later lab: the Lab 1 Brief, Section 2, `docs/lab1/lab1_brief.pdf`.

[13] `src/libipc/include/sentinel/ipc.h`, in your repository. It states what each function of Lab 3 does, in order, with each of its errors and its log lines.

[14] The make targets of all labs, such as `make test` and `make watch`: the SENTINEL Lab Manual, Section 4, `docs/manual/sentinel_manual.pdf`.

[15] `make submit` builds your work from clean, tests it, and writes the zip. The Submission Guide, Section 5, `docs/submit/submit.pdf`, describes it, and Section 6 the upload.

[16] Your own tests are C programs with `assert()` and no framework. The Lab 1 Brief, Section 4.2, `docs/lab1/lab1_brief.pdf`, states their rules.

6 Hand it in

The hand-in of Lab 3 is your code and three files of your own runs, in `docs/lab3/`. `make submit LAB=3` puts them in the zip. Make each file with your own library, at the top of your repository.

6.1 The record runs

First the control: one worker, on one processor, for 20 s. Then 8 workers, with each processor. The second run needs two processors or more: `nproc` gives the number. Then the same 8 workers with ThreadSanitizer, for 5 s. The third run takes about 12 s, and ThreadSanitizer ends it with exit status 66 when it finds a race.

```
make
SENTINEL_PROFILE=world build/demo_lab3 --workers 1 --ms 20000 \
    --log build/lab3_1.log > docs/lab3/demo_1worker.txt
SENTINEL_PROFILE=world build/demo_lab3 --workers 8 --cpus all --ms 20000 \
    --log build/lab3_8.log > docs/lab3/demo_8workers.txt
make lab3-tsan
SENTINEL_PROFILE=world build/demo_lab3_tsan --workers 8 --cpus all --ms 5000 \
    --log build/lab3_tsan.log > /dev/null 2> docs/lab3/tsan.txt
```

This is a part of `demo_8workers.txt` from one run of 10 s:

```
  processed         20722 of 20705 queued
  picture counted   20722 reports, 36 tracks (36 walked)
    accounted for                 20722   DOES NOT BALANCE -- nothing here is protected (R4)
  shared data DISAGREES: unprotected workers lost or repeated updates (Part 4 fixes this)
```

The workers applied 17 reports more than the centre queued: two workers took the same report. In other runs the workers did not stop, and the demo printed `queue says 18446744073709112535 reports still wait`. This is the start of one report in `tsan.txt`, with the addresses of the program removed. Line 88 of that `queue.c` is `q->popped++`, which two workers ran at the same time:

```
WARNING: ThreadSanitizer: data race (pid=50467)
  Read of size 8 at 0xfffffffffc940 by thread T5:
    #0 queue_pop src/libipc/src/queue.c:88
    #1 worker_main src/libipc/src/pool.c:78

  Previous write of size 8 at 0xfffffffffc940 by thread T4:
    #0 queue_pop src/libipc/src/queue.c:88
    #1 worker_main src/libipc/src/pool.c:78
```

6.2 The checks of agreement

The marker compares the three files, and your code:

- `demo_1worker.txt` has the line `workers` with 1, a LOSS LEDGER that says `BALANCED`, and `shared data agrees`.
- `demo_8workers.txt` has the line `workers` with 8. It ends with `shared data DISAGREES` or `THE WORKERS DID NOT STOP`. If it says `agrees`, run it again: on two processors or more, each of 20 measured runs showed the race.
- `tsan.txt` has at least one report that names your `src/queue.c` and one that names your `src/track.c`. The line that each report names is a line of your code that reads or writes a shared field. A report on `stop` in `pool.c`, or on a flag in `given/pool_watch.c`, is expected too.

The oral exam asks you why the totals of the second file disagree, with your own files. Then run `make submit LAB=3`, and upload the zip.[17]

7 Done when, and the marks

☐ `make` prints no warning, and `make test`[18] passes your two tests.

[17] `make submit` builds your work from clean, tests it, and writes the zip. The Submission Guide, Section 5, `docs/submit/submit.pdf`, describes it, and Section 6 the upload.

[18] The make targets of all labs, such as `make test` and `make watch`: the SENTINEL Lab Manual, Section 4, `docs/manual/sentinel_manual.pdf`.

☐ `make accept3` gives 23 of 23.
☐ The three files of Section 6 are in `docs/lab3/`, and they agree as Section 6.2 states.
☐ `valgrind` reports no error and no leak.
☐ Your code has no lock, no atomic variable and no semaphore.
☐ The conditions of a finished Lab 1 in the Lab 1 Brief[19] still hold, and `make accept2` still gives 26 of 26.

Part	**Marks**
Acceptance test (`make accept3`), in proportion to the checks that pass	20
Course tests: the tests of the course, on each function of `ipc.h`[20] that you write	24
Your two tests (Section 5)	12
Code rules, and the errors of each call (Section 4)	12
`valgrind`: no error and no leak; each thread joined	4
Agreement of the hand-in files (Section 6.2)	8
Total	**80**

You do not get the course tests. They check only what `ipc.h` and this brief state. The oral exam has its own 20 marks.

[19] The conditions of a finished Lab 1, such as no warning and a clean `valgrind`: the Lab 1 Brief, Section 5, `docs/lab1/lab1_brief.pdf`.
[20] `src/libipc/include/sentinel/ipc.h`, in your repository. It states what each function of Lab 3 does, in order, with each of its errors and its log lines.